

Informe caso de estudio EcoMarket

Nicolas Rojas

ADSO

2025

A mi me toco el modulo del carrito de compras y pagos en línea. El sistema deberá permitir al usuario que agregue la cantidad de productos que desee a una lista y finalmente mostrar un resumen completo de los productos que agrego y el total a pagar.

Para este proyecto usare las siguientes herramientas:

-Lenguaje de Programación: Java.

Entorno de Desarrollo (IDE):NetBeans.

Clases de Interfaz: javax.swing.JOptionPane para interactuar con el usuario (pedir datos y mostrar resultados).

Estructura de Datos: java.util.ArrayList (usando la interfaz List para la n cantidad de productos que el

usuario quiere poner y no poner 1 por 1 los productos como objetos).

Estrategia que voy a seguir para completar el módulo:

1. crear la clase donde irán los productos que se agreguen al carrito (Productos.java).

- **Atributos:** id, nombre, precio, descripción, y reseña.
- **Constructores:** usare 2 constructores: el primero con todos los atributos asignados y el segundo sin la reseña porque tal vez el usuario no quiera poner ninguna reseña.
- **Métodos:**
 - Métodos Getter y Setter: Para acceder y modificar de forma controlada los atributos del producto (por ejemplo, setPrecio() para actualizar el valor).
 - toString(): Se implementará para generar una cadena de texto ordenada y fácil de leer, que contendrá todos los detalles de un producto.

2. crear la clase donde ira la información del usuario (Usuario.java)

- **Función de la Clase Usuario**

Simular el perfil de un cliente, almacenar su historial de compras y generar sugerencias de productos.

- **Historial de Compras:** Almacena todos los objetos Productos que el usuario ha comprado exitosamente. Esto simula una base de datos de transacciones.
- **Motor de Recomendaciones (Básico):** Analiza el historial guardado para sugerir un producto relacionado, mejorando la experiencia del cliente.

La implementación se basará en el manejo de una (List) y la aplicación de lógica de flujo (if-else).

Implementación de la creación del objeto usuario:

Para manejar un número variable de productos comprados, se utilizó un ArrayList de Java:

- **Atributo:** Se declara el atributo historialCompra como un List<Productos>. Esto permite que el historial sea una colección dinámica.

- Constructor: El constructor public Usuario() se encarga de iniciar la lista: `this.historialCompra = new ArrayList<>()`; Esto asegura que el historial esté vacío y listo para usarse desde el momento en que se crea el objeto Usuario.

- **Método agregarHiistorial(Productos p):**

Es el que va guardar la compra cuando la simulación de pago en Principal.java es exitosa.

- **Método generarRecomendaciones():**

Este es el método principal donde me base en el principio de "Si compre X producto, te recomendamos algo de la misma categoría."

3.clase principal(Principal.java):

Esta ser el main y en donde se ejecutara el sistema.

Implementación del Carrito:

Se declarará una lista dinámica: `List<Productos>`
`carrito = new ArrayList<>();`. Esta lista nos permitirá agregar n cantidad de productos.

Proceso de interacción:

- Se implementará un bucle while controlado por una variable booleana (agregarMas).
- Dentro del bucle, se solicitarán los datos del producto (nombre, precio, etc.) usando `JOptionPane.showInputDialog()`.
- Una vez validados los datos, se creará un objeto `Productos` y se agregará al `ArrayList` usando el método `.add()`.
- Al final de cada iteración, se preguntará al usuario si desea continuar agregando.

Implementación del proceso de pago:

Utilicé el `JOptionPane.showInputDialog()` para solicitar al usuario que ingrese su método de pago.

- **Formato del Total:** Use `String.format("$%.3f", total)` para mostrar el valor exacto que se va a pagar.
- **Almacenamiento:** La respuesta del usuario se guarda en la variable `metodoPago` de tipo `String`.
- **Validación del pago(if-else):** aquí es donde se simula el pago exitoso o no.

Implementación de recomendaciones:

Esta sección se ejecuta siempre después de intentar el pago .

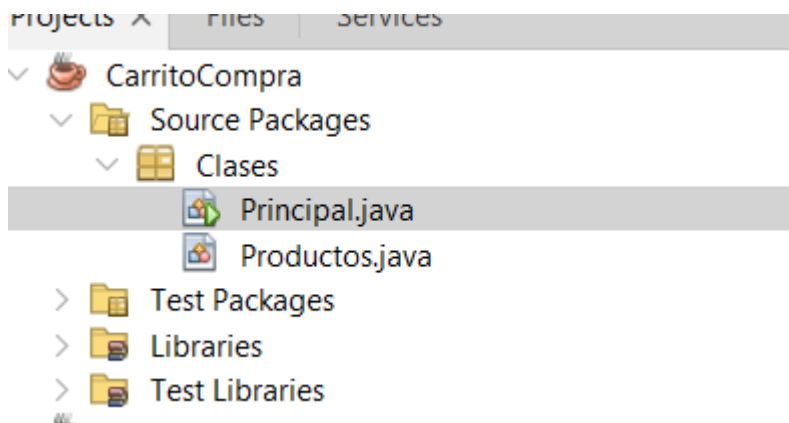
- **Generación de la Sugerencia:** Se llama al método `generarRecomendaciones()` del objeto `usuarioActual`. Este método decide qué recomendar. La recomendación se guarda en la variable `recomendacion`.

- Finalmente, se usa otro JOptionPane para mostrar el mensaje de recomendación al usuario.

Informe final:

- Una vez que el usuario finalice su compra, se recorrerá todo el ArrayList usando un bucle for-each.
- Se concatenará la información de cada producto (obtenida gracias a producto.toString()) en una sola cadena.
- Se calculará la suma total de los precios usando esto:(total += producto.getPrecio()).
- El informe final (resumen del carrito y total a pagar) se mostrará en un único JOptionPane.showMessageDialog().

Avances del sistema:



```
package Clases;

public class Productos {
    public int id;
    public String reseña="";
    public String nombre="";
    public double precio;
    public String descripcion="";

    //constructor principal
    public Productos(int id, String reseña, String nombre, double precio, String descripcion){
        this.id=id;
        this.reseña=reseña;
        this.nombre=nombre;
        this.precio=precio;
        this.descripcion=descripcion;
    }

    //constructor sin reseñas en caso de que el usuario no haga reseñas

    public Productos(int id, String nombre, double precio, String descripcion){
        this.id=id;
        this.reseña="sin reseña";
        this.nombre=nombre;
        this.precio=precio;
        this.descripcion=descripcion;
    }

    public int getId() {
        return id;
    }
}
```



```
public void setId(int id) {  
    this.id = id;  
}
```

```
public String getReseña() {  
    return reseña;  
}
```

```
public void setReseña(String reseña) {  
    this.reseña = reseña;  
}
```

```
public String getNombre() {  
    return nombre;  
}
```

```
public void setNombre(String nombre) {  
    this.nombre = nombre;  
}
```

```
public double getPrecio() {  
    return precio;  
}
```

```
public void setPrecio(double precio) {  
    this.precio = precio;  
}
```

```
public String getDescripcion() {  
    return descripcion;  
}
```

```
public void setDescripcion(String descripcion) {  
    this.descripcion = descripcion;  
}
```

```
public void setDescription(String descripcion) {
    this.descripcion = descripcion;
}

//metodo para mostrar la informacion

@Override
public String toString(){
    return("\n id: "+id+"\n comentarios: "+reseña+ "\n nombre: "+nombre+ "\n precio $:"+precio +
        "\n descripcion: "+descripcion);
}

}
```